Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 3

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

« Розробка інтерфейсу користувача »

Виконав:

студент групи ІМ-53
Брагар Артем Сергійович
Номер в списку - 2

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі розробки новий проект з ім’ям Lab3.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Завдання згідно варіанту:

Номер у списку Жлаб2 = 2, тому для цієї роботи Ж = Жлаб2 + 1 = 3.

1. Масив вказівників на Shape: динамічний масив (3 mod 3 = 0) обсягом 103 об’єкти (3 + 100).

2. «Гумовий» слід: пунктирна лінія чорного кольору (3 mod 4 = 3).

3. Прямокутник: ввід від центру до одного з кутів (3 mod 2 = 1); відображення – чорний контур без заповнення (3 mod 5 = 3).

4. Еліпс: ввід по двох протилежних кутах охоплюючого прямокутника (3 mod 2 = 1); відображення – чорний контур з кольоровим заповненням (3 mod 5 = 3); колір заповнення – блакитний (3 mod 6 = 3).

5. Позначка поточного типу об’єкта: в заголовку вікна (3 mod 2 = 1).

Вимоги до інтерфейсу:

1. Усі методи-обробники повідомлень (у C# – обробники подій) повинні бути методами класів.

2. Для вибору типу об’єкта повинна бути панель інструментів (Toolbar) з кнопками-іконками відповідно до типів об’єктів. Кнопки дублюють підпункти меню «Об’єкти» і мають підказки (tooltips).

3. Меню «Об’єкти» розташоване праворуч від меню «Файл» та ліворуч від меню «Довідка». Підпункти меню містять назви геометричних форм українською мовою.

Файли лабораторної

Github
Program.cs

```
namespace Lab3;
```

```
static class Program
{
    /// <summary>
    ///  The main entry point for the application.
    /// </summary>
    [STAThread]
    static void Main()
    {
        // To customize application configuration such as set high DPI
settings or default font,
        // see https://aka.ms/applicationconfiguration.
        ApplicationConfiguration.Initialize();
        Application.Run(new Form1());
    }
}
```

Form1.cs

```
namespace Lab3;


public partial class Form1 : Form
{
    private Editor editor;
    private void SelectShapeType(Editor.ShapeType type)
    {
        switch (type)
        {
            case Editor.ShapeType.Point:
            Text = "Lab3-Крапка";
            break;
            case Editor.ShapeType.Rectangle:
            Text = "Lab3-Прямокутник";
            break;
            case Editor.ShapeType.Line:
            Text = "Lab3-Лінія";
            break;
```

```
            case Editor.ShapeType.Ellipse:
                Text = "Lab3-Еліпс";
                break;
        }
        editor.SelectShapeType(type);

    }
    private void CreateMenu()
    {
        MenuStrip menuStrip = new MenuStrip();
        ToolStripMenuItem fileMenu = new ToolStripMenuItem("Файл");
        ToolStripMenuItem objectsMenu = new
ToolStripMenuItem("Об'єкти");
        ToolStripMenuItem helpMenu = new ToolStripMenuItem("Довідка");
        ToolStripMenuItem pointItem = new ToolStripMenuItem("Крапка");
        ToolStripMenuItem rectItem = new
ToolStripMenuItem("Прямокутник");
        ToolStripMenuItem lineItem = new ToolStripMenuItem("Лінія");
        ToolStripMenuItem ellipseItem = new ToolStripMenuItem("Еліпс");

        Controls.Add(menuStrip);
        MainMenuStrip = menuStrip;
        menuStrip.Items.Add(fileMenu);
        menuStrip.Items.Add(objectsMenu);
        menuStrip.Items.Add(helpMenu);
        objectsMenu.DropDownItems.Add(pointItem);
        objectsMenu.DropDownItems.Add(rectItem);
        objectsMenu.DropDownItems.Add(lineItem);
        objectsMenu.DropDownItems.Add(ellipseItem);

        pointItem.Tag = Editor.ShapeType.Point;
        lineItem.Tag = Editor.ShapeType.Line;
        rectItem.Tag = Editor.ShapeType.Rectangle;
        ellipseItem.Tag = Editor.ShapeType.Ellipse;
```

```
        pointItem.Click += ShapeSelector_Click;
        lineItem.Click += ShapeSelector_Click;
        rectItem.Click += ShapeSelector_Click;
        ellipseItem.Click += ShapeSelector_Click;
        fileMenu.Click += FileMenu_Click;
        helpMenu.Click += HelpMenu_Click;
    }
    private void CreateShapesBar()
    {
        ToolStrip shapesBar = new ToolStrip();
        shapesBar.ImageScalingSize = new Size(24, 24);
        ToolStripButton pointButton = new ToolStripButton();
        ToolStripButton rectButton = new ToolStripButton();
        ToolStripButton lineButton = new ToolStripButton();
        ToolStripButton ellipseButton = new ToolStripButton();

        pointButton.ToolTipText = "Крапка";
        rectButton.ToolTipText = "Прямокутник";
        lineButton.ToolTipText = "Лінія";
        ellipseButton.ToolTipText = "Еліпс";

        pointButton.Tag = Editor.ShapeType.Point;
        rectButton.Tag = Editor.ShapeType.Rectangle;
        lineButton.Tag = Editor.ShapeType.Line;
        ellipseButton.Tag = Editor.ShapeType.Ellipse;

        pointButton.Click += ShapeSelector_Click;
        rectButton.Click += ShapeSelector_Click;
        lineButton.Click += ShapeSelector_Click;
        ellipseButton.Click += ShapeSelector_Click;

        pointButton.Image = IconShapes.CreatePointIcon();
        rectButton.Image = IconShapes.CreateRectIcon();
        lineButton.Image = IconShapes.CreateLineIcon();
        ellipseButton.Image = IconShapes.CreateEllipseIcon();
```

```
        Controls.Add(shapesBar);
        shapesBar.Items.Add(pointButton);
        shapesBar.Items.Add(rectButton);
        shapesBar.Items.Add(lineButton);
        shapesBar.Items.Add(ellipseButton);
    }
    public Form1()
    {
        InitializeComponent();
        ClientSize = new Size(1080, 720);
        this.DoubleBuffered = true;
        editor = new Editor();
        CreateShapesBar();
        CreateMenu();
        SelectShapeType(Editor.ShapeType.Point);

        this.MouseDown += Form_MouseDown;
        this.MouseMove += Form_MouseMove;
        this.MouseUp += Form_MouseUp;
        this.Paint += Form_Paint;
    }
    private void Form_MouseDown(object? sender, MouseEventArgs e)
    {
        editor.OnMouseDown(e.Location);
    }
    private void Form_MouseMove(object? sender, MouseEventArgs e)
    {
        if (editor.OnMouseMove(e.Location))
        {
            Invalidate();
        }
    }
    private void Form_MouseUp(object? sender, MouseEventArgs e)
    {
```

```
        switch (editor.OnMouseUp())
        {
            case Editor.OnMouseUpResult.ShapeAdded:
                Invalidate();
            break;
            case Editor.OnMouseUpResult.ArrayIsFull:
                MessageBox.Show("Shapes Array is full");
                Invalidate();
            break;
        }
    }
    private void Form_Paint(object? sender, PaintEventArgs e)
    {
        Graphics g = e.Graphics;
        editor.DrawShapes(g);
    }
    private void ShapeSelector_Click(object? sender, EventArgs e)
    {
        ToolStripItem item = (ToolStripItem)sender!;
        Editor.ShapeType type = (Editor.ShapeType)item.Tag!;
        SelectShapeType(type);
    }
    private void FileMenu_Click(object? sender, EventArgs e)
    {
        MessageBox.Show("there is something, but not here");
    }
    private void HelpMenu_Click(object? sender, EventArgs e)
    {
        MessageBox.Show("Choose Objects option to draw some shapes");
    }
}
```

Editor.cs

```
namespace Lab3;
```

```
using System.Drawing.Drawing2D;
public class Editor
{
    public enum ShapeType
    {
        Point,
        Line,
        Rectangle,
        Ellipse
    }
    public enum OnMouseUpResult
    {
        ShapeAdded,
        ArrayIsFull,
        NothingHappend
    }
    private ShapeType currentShapeType = ShapeType.Point;
    private const int MaxShapes = 103;
    private Shape[] shapes;
    public Editor()
    {
        shapes = new Shape[MaxShapes];
    }
    private int shapeCount = 0;
    private bool isDrawing = false;
    private Point currentStart;
    private Point currentEnd;
    public void OnMouseDown(Point point)
    {
        isDrawing = true;
        currentStart = point;
        currentEnd = point;
    }
    public bool OnMouseMove(Point point)
    {
```

```
        if (isDrawing)
        {
            currentEnd = point;
        }
        return isDrawing;
    }
    public OnMouseUpResult OnMouseUp()
    {
        if (!isDrawing)
        {
            return OnMouseUpResult.NothingHappend;
        }

        isDrawing = false;

        if (shapeCount >= shapes.Length)
        {
            return OnMouseUpResult.ArrayIsFull;
        }
        Shape newShape = CreateShape();
        newShape.StartPoint = currentStart;
        newShape.EndPoint = currentEnd;
        shapes[shapeCount] = newShape;
        shapeCount++;
        return OnMouseUpResult.ShapeAdded;
    }
    private Shape CreateShape()
    {
        Shape newShape;
        switch (currentShapeType)
        {
            case ShapeType.Point:
                newShape = new PointShape();
            break;
            case ShapeType.Line:
```

```
                newShape = new LineShape();
            break;
            case ShapeType.Rectangle:
                newShape = new RectShape();
            break;
            case ShapeType.Ellipse:
                newShape = new EllipseShape();
            break;
            default:
                throw new InvalidOperationException("unknown shape");
        }
        return newShape;
    }
    public void DrawShapes(Graphics g)
    {
        for (int i = 0; i < shapeCount; i++)
        {
            shapes[i].Show(g);
        }
        if (isDrawing)
        {
            DrawTemp(g);
        }
    }
    private void DrawTemp(Graphics g)
    {
        Shape previewShape = CreateShape();
        previewShape.IsPreview = true;
        previewShape.StartPoint = currentStart;
        previewShape.EndPoint = currentEnd;
        previewShape.OutlinePen.DashStyle = DashStyle.Dash;
        previewShape.Show(g);
    }
    public  void SelectShapeType(ShapeType type)
    {
```

```
        currentShapeType = type;
    }
}
```

IconShapes.cs

```
namespace Lab3;
static class IconShapes
{
    public static Bitmap CreatePointIcon()
    {
        Bitmap pointBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(pointBmp))
        using (Brush brush = new SolidBrush(Color.Black))
        {
            g.FillEllipse(brush, 8, 8, 8, 8);
        }
        return pointBmp;
    }
    public static Bitmap CreateRectIcon()
    {
        Bitmap rectBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(rectBmp))
        using (Pen pen = new Pen(Color.Black))
        {
            g.DrawRectangle(pen, 4, 6, 16, 12);
        }
        return rectBmp;
    }
    public static Bitmap CreateEllipseIcon()
    {
        Bitmap ellipseBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(ellipseBmp))
        using (Pen pen = new Pen(Color.Black))
        {
```

```
            g.DrawEllipse(pen, 2, 5, 20, 14);
        }
        return ellipseBmp;
    }
    public static Bitmap CreateLineIcon()
    {
        Bitmap lineBmp = new Bitmap(24,24);
        using (Graphics g = Graphics.FromImage(lineBmp))
        using (Pen pen = new Pen(Color.Black))
        {
            g.DrawLine(pen, 3, 3, 20, 20);
        }
        return lineBmp;
    }
}
```

Shapes/PointShape.cs

```
namespace Lab3;
public class PointShape : Shape
{
    public Brush FillBrush;
    public PointShape()
    {
        FillBrush = new SolidBrush(OutlinePen.Color);
    }
    public override void Show(Graphics g)
    {
        int width = 4;
        int height = 4;
        int x = StartPoint.X - (width / 2);
        int y = StartPoint.Y - (height / 2);
        g.FillEllipse(FillBrush, x, y, width, height);
    }
}
```

Shapes/LineShape.cs

```
namespace Lab3;
public class LineShape : Shape
{
    public override void Show(Graphics g)
    {
        g.DrawLine(OutlinePen, StartPoint, EndPoint);
    }
}
```

Shapes/RectShape.cs

```
namespace Lab3;
public class RectShape : Shape
{
    public override void Show(Graphics g)
    {
        int dx = Math.Abs(EndPoint.X - StartPoint.X);
        int dy = Math.Abs(EndPoint.Y - StartPoint.Y);
        int width = dx * 2;
        int height = dy * 2;
        int x = StartPoint.X - dx;
        int y = StartPoint.Y - dy;
        g.DrawRectangle(OutlinePen, x, y, width, height);
    }
}
```

Shapes/EllipseShape.cs

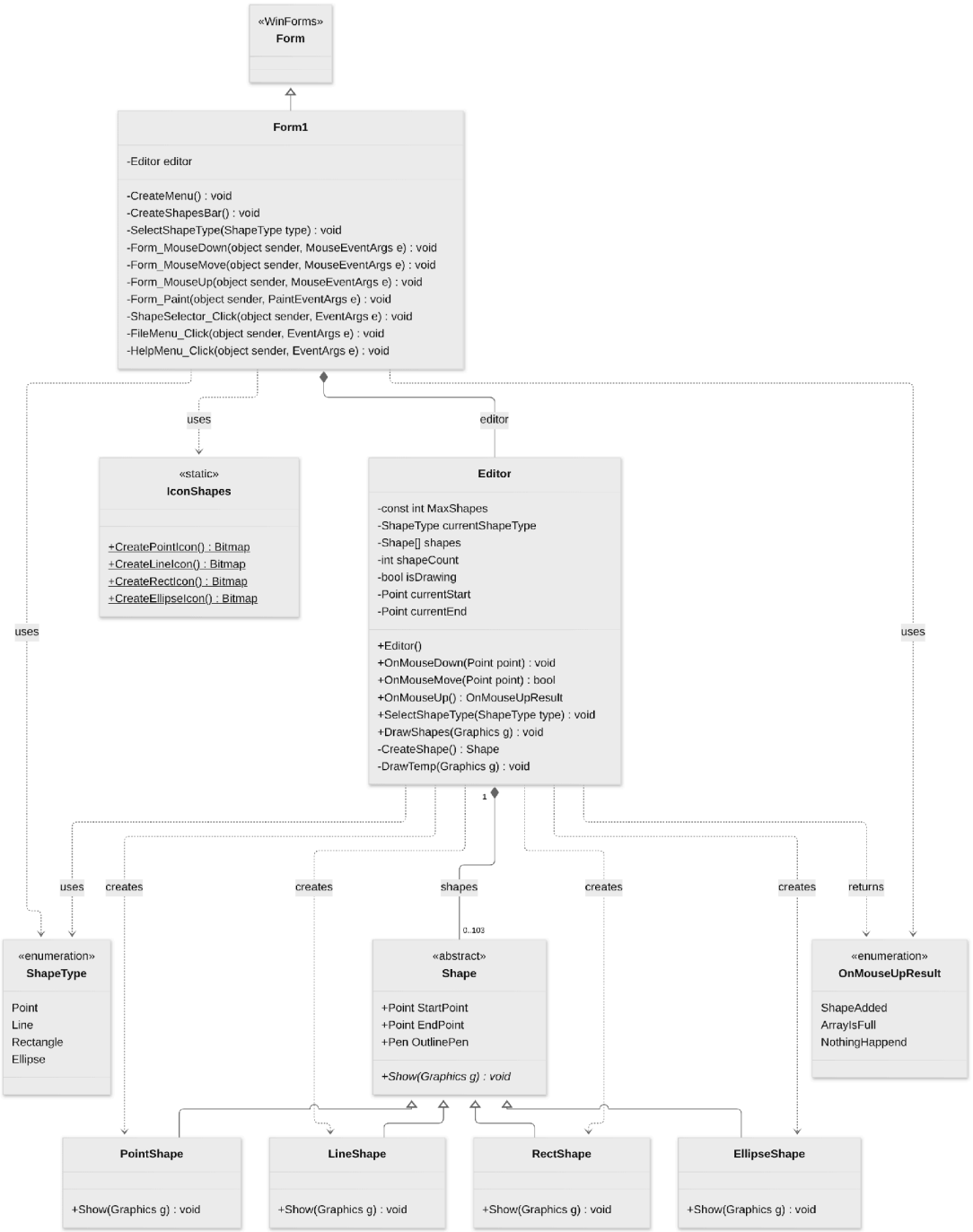
```
namespace Lab3;
public class EllipseShape : Shape
{
    public Brush FillBrush = new SolidBrush(Color.Cyan);
    public override void Show(Graphics g)
    {
        int radX = Math.Abs(EndPoint.X - StartPoint.X);
        int radY = Math.Abs(EndPoint.Y - StartPoint.Y);
        int x = Math.Min(StartPoint.X, EndPoint.X);
        int y = Math.Min(StartPoint.Y, EndPoint.Y);
```

```
        if(!IsPreview) {g.FillEllipse(FillBrush, x, y, radX, radY);}
        g.DrawEllipse(OutlinePen, x, y, radX, radY);
    }
}
```

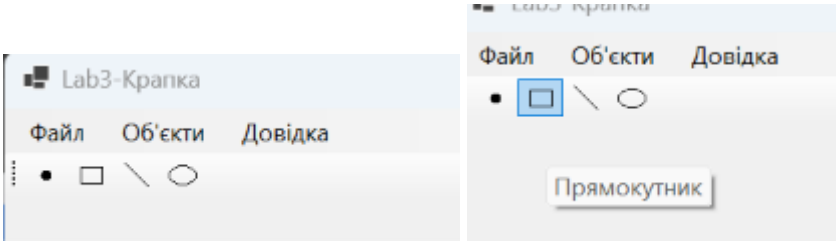
Shapes/Shape.cs

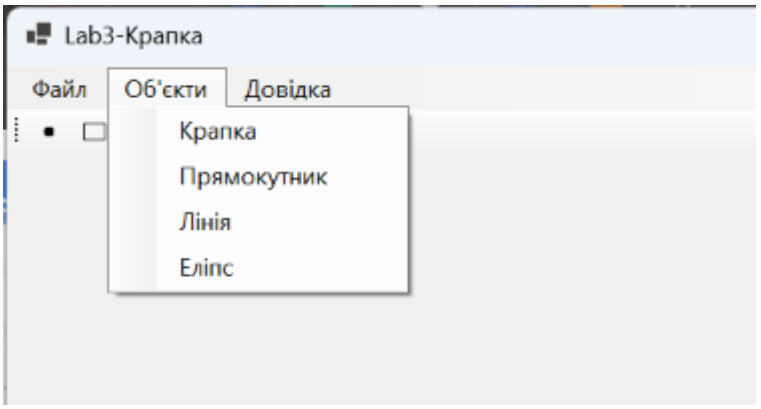
```
namespace Lab3;
public abstract class Shape
{
    public Point StartPoint = new Point();
    public Point EndPoint = new Point();
    public Pen OutlinePen = new Pen(Color.Black);
    public bool IsPreview {get; set;}
    public abstract void Show(Graphics g);
}
```

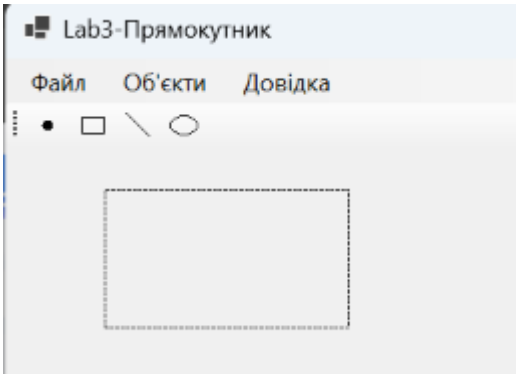
Діаграма класів

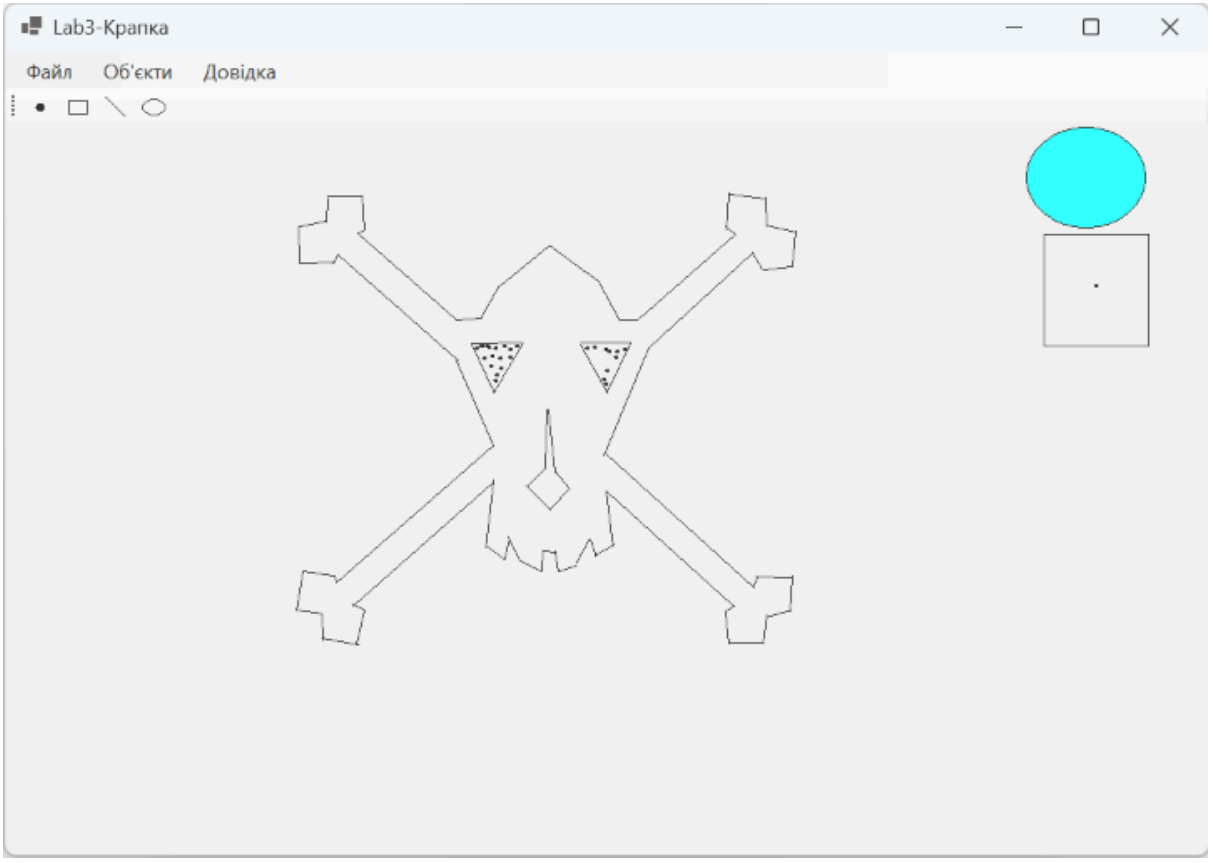


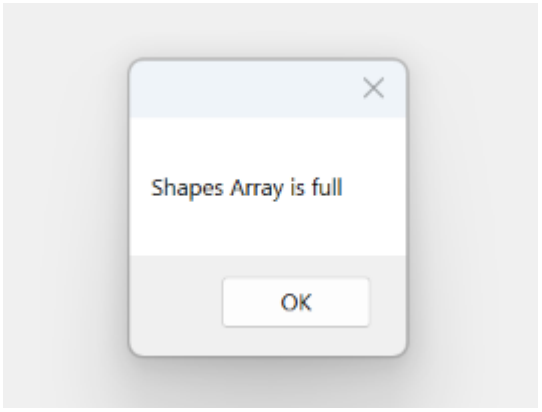
Скріншоти


Lab3-Крапка
Файл
Об'єкти
Довідка
Файл
Об'єкти
Довідка
Прямокутник


Lab3-Крапка
Файл
Об'єкти
Довідка
Крапка
Прямокутник
Лінія
Еліпс


Lab3-Прямокутник
Файл
Об'єкти
Довідка


Lab3-Крапка
Файл
Об'єкти
Довідка


Shapes Array is full
OK

Висновки:

У ході лабораторної роботи я розробив графічний інтерфейс користувача для графічного редактора об’єктів на C# (WinForms). Для варіанта Ж = 3 реалізовано динамічний масив на 103 фігури, «гумовий» слід у вигляді чорної пунктирної лінії, ввід прямокутника від центру, ввід еліпса по двох кутах охоплюючого прямокутника з блакитним заповненням та позначку поточного типу фігури в заголовку вікна.

Вибір типу фігури продубльовано панеллю інструментів ToolStrip: кнопки мають іконки, намальовані програмно за допомогою Bitmap і Graphics (клас IconShapes), та підказки (ToolTipText). Меню і кнопки обслуговує один обробник ShapeSelector_Click, який отримує тип фігури з властивості Tag, тому вони завжди відповідають одне одному. Усі обробники подій є методами класів.

Логіку редактора (масив фігур, стан введення, малювання) винесено з Form1 в окремий клас Editor, а форма лише передає йому події та перемальовує вікно, що відповідає принципу єдиної відповідальності. Поліморфізм використано під час малювання: Editor викликає метод Show() для кожної фігури через посилання на абстрактний клас Shape.

Таким чином я отримав вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши графічний інтерфейс користувача.